Microservicios y APIs Escalables

Universidad de Palermo · 2026

Sprint 11

Comunicación entre Microservicios con OpenFeign

Clínica Veterinaria "Patitas Felices"

Contexto y objetivo del Sprint

Al terminar el Sprint 10, los cuatro microservicios (ms-dueno, ms-mascota, ms-veterinario, ms-turno) están funcionando de forma **independiente**: arrancan, se registran en Eureka, reciben su configuración del Config Server y exponen endpoints REST.

El problema pendiente es la **validación cruzada**: cuando se crea un turno se debe verificar que la mascota y el veterinario existan — pero esa información vive en *otros microservicios*. En el monolito era una llamada a un repository; aquí es una llamada HTTP entre servicios.

Objetivo: configurar Spring Cloud OpenFeign en ms-turno para consumir ms-mascota y ms-veterinario a través de Eureka. Al final del sprint ms-turno validará la existencia de mascota y veterinario antes de confirmar un turno, usando el nombre de servicio registrado en Eureka — sin hardcodear IPs ni puertos.

¿Por qué OpenFeign y no RestTemplate o WebClient?

Opción	Estilo	Integración Eureka	Uso recomendado
RestTemplate	Imperativo / verboso	Manual (ribbon/LoadBalancer)	Legacy — no recomendado en SB 4
WebClient	Reactivo (Flux/Mono)	Con anotaciones extra	APIs reactivas / alta concurrencia
OpenFeign	Declarativo (interfaz)	Automático con @FeignClient	Microservicios síncronos — este curso

> Con OpenFeign se define una interfaz Java anotada: Feign genera el cliente HTTP y el **Load Balancer** de Spring Cloud resuelve `lb://ms-mascota` contra el registro de Eureka en cada llamada.

Flujo de validación al crear un turno

```
POST /api/turnos
      |
      ▼
[TurnoController]
      |
      ▼
```

```
[TurnoServiceImpl]
      |── MascotaFeignClient.existsById(mascotaId)  ──→  ms-mascota :8082
      |── VeterinarioFeignClient.existsById(vetId)  ──→  ms-veterinario :8083
      |
      ▼  (ambos existen → continuar)
[TurnoRepositoryPort] → BD de ms-turno
```

⚠️ Eureka resuelve "ms-mascota" → IP:puerto real. Si el servicio escala a 3 instancias, el Load Balancer las rota automáticamente.

Pasos del Sprint

PASO 1 — Agregar dependencia OpenFeign en ms-turno

Abrir `ms-turno/pom.xml` y agregar dentro de `<dependencies>`:

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

Verificar que la versión la resuelve el BOM de Spring Cloud (2025.0.0) declarado en el pom padre — no agregar versión explícita.

PASO 2 — Habilitar Feign en la clase principal de ms-turno

Abrir `MsTurnoApplication.java` y agregar la anotación `@EnableFeignClients` sobre `@SpringBootApplication`:

```
@SpringBootApplication
@EnableFeignClients
public class MsTurnoApplication { ... }
```

Sin @EnableFeignClients Spring Boot no escanea las interfaces @FeignClient y el contexto no arranca.

PASO 3 — Crear el cliente Feign para ms-mascota

Crear la interfaz en el paquete `com.patitas.msturno.adapters.out.feign`:

```
package com.patitas.msturno.adapters.out.feign;

import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

@FeignClient(name = "ms-mascota")
public interface MascotaFeignClient {

    @GetMapping("/api/mascotas/{id}",
                produces = "application/json")
    MascotaResponse findById(@PathVariable("id") Long id);
}
```

Nombre en @FeignClient: debe coincidir exactamente con `spring.application.name` del microservicio destino tal como aparece en Eureka. Spring Cloud resuelve `lb://ms-mascota` automáticamente.

Crear también la clase de respuesta reducida `MascotaResponse` en el mismo paquete — solo los campos que ms-turno necesita verificar:

```
package com.patitas.msturno.adapters.out.feign;

import lombok.Data;

@Data
public class MascotaResponse {
    private Long id;
    private String nombre;
    private Long duenoId;
}
```

⚠️ NO reutilizar la clase Mascota de ms-mascota importando ese módulo — los microservicios NO comparten código de dominio. Cada uno define su propia clase DTO de consumo.

PASO 4 — Crear el cliente Feign para ms-veterinario

Mismo paquete `adapters.out.feign`:

```
@FeignClient(name = "ms-veterinario")
public interface VeterinarioFeignClient {

    @GetMapping("/api/veterinarios/{id}",
                produces = "application/json")
    VeterinarioResponse findById(@PathVariable("id") Long id);
}
```

Y la clase reducida `VeterinarioResponse`:

```
@Data
public class VeterinarioResponse {
    private Long id;
    private String nombre;
    private String apellido;
    private String especialidad;
}
```

PASO 5 — Crear puertos salientes Feign en el dominio de ms-turno

Siguiendo la arquitectura hexagonal, el dominio no puede depender directamente de `MascotaFeignClient` (que es un adaptador). Se definen puertos salientes en `com.patitas.msturno.domain.ports.out`:

```
// Puerto saliente — solo una operación
public interface MascotaServicePort {
    boolean existsById(Long id);
}

public interface VeterinarioServicePort {
    boolean existsById(Long id);
}
```

El dominio usa estos puertos. Los adaptadores `MascotaFeignAdapter` y `VeterinarioFeignAdapter` implementan los puertos usando los Feign clients (ver paso 6).

PASO 6 — Implementar los adaptadores Feign

En `adapters.out.feign`:

```
@Component
@RequiredArgsConstructor
public class MascotaFeignAdapter implements MascotaServicePort {

    private final MascotaFeignClient client;

    @Override
    public boolean existsById(Long id) {
        try {
            client.findById(id);
            return true;
        } catch (feign.FeignException.NotFound e) {
            return false;
        }
    }
}
```

Repetir la misma estructura para `VeterinarioFeignAdapter implements VeterinarioServicePort`.

> La estrategia try/catch sobre **`FeignException.NotFound`** convierte el HTTP 404 del servicio remoto en un booleano limpio. El dominio no sabe nada de HTTP.

PASO 7 — Modificar TurnoServiceImpl para usar los puertos

En `TurnoServiceImpl` inyectar los dos puertos nuevos (ya que `@RequiredArgsConstructor` los resuelve vía constructor):

```
@Service
@RequiredArgsConstructor
public class TurnoServiceImpl implements TurnoService {

    private final TurnoRepositoryPort turnoRepository;
    private final MascotaServicePort  mascotaService;
    private final VeterinarioServicePort veterinarioService;

    @Override
    public Turno save(Turno turno) {
        if (!mascotaService.existsById(turno.getMascotaId()))
            throw new MascotaNotFoundException(turno.getMascotaId());

        if (!veterinarioService.existsById(turno.getVeterinarioId()))
            throw new VeterinarioNotFoundException(turno.getVeterinarioId());

        turno.setEstado(EstadoTurno.PENDIENTE);
        return turnoRepository.save(turno);
    }
    // findAll, findById, update, deleteById — sin cambios
}
```

Crear las excepciones de dominio `MascotaNotFoundException` y `VeterinarioNotFoundException` en `domain.exceptions` (igual que `DuenoNotFoundException` en ms-dueno — sin imports de Spring/HTTP).

PASO 8 — Actualizar GlobalExceptionHandler de ms-turno

Agregar los nuevos handlers en `GlobalExceptionHandler` de ms-turno:

```
    @ExceptionHandler(MascotaNotFoundException.class)
    public ResponseEntity<Map<String,Object>> handleMascota(MascotaNotFoundException ex)
{
        return buildError(HttpStatus.NOT_FOUND, ex.getMessage());
    }

    @ExceptionHandler(VeterinarioNotFoundException.class)
    public ResponseEntity<Map<String,Object>> handleVet(VeterinarioNotFoundException ex)
{
        return buildError(HttpStatus.NOT_FOUND, ex.getMessage());
    }

    // Capturar errores de Feign genéricos (servicio caído, timeout, etc.)
    @ExceptionHandler(feign.FeignException.class)
    public ResponseEntity<Map<String,Object>> handleFeign(feign.FeignException ex) {
        return buildError(HttpStatus.SERVICE_UNAVAILABLE,
            "Servicio remoto no disponible: " + ex.getMessage());
    }
```

PASO 9 — Configurar timeout y logging de Feign

Agregar en `ms-turno.yml` del repositorio `patitas-config`:

```
    feign:
      client:
        config:
          default:
            connectTimeout: 3000   # ms para conectar
            readTimeout: 5000      # ms para leer respuesta
      compression:
        request:
          enabled: true
        response:
          enabled: true

    logging:
      level:
        com.patitas.msturno.adapters.out.feign: DEBUG
```

⚠️ Sin timeout, si ms-mascota se cae, ms-turno queda bloqueado indefinidamente esperando respuesta. Siempre definir timeouts en comunicación entre servicios.

PASO 10 — Prueba end-to-end del flujo completo

Orden de arranque: config-server → eureka-server → ms-mascota → ms-veterinario → ms-turno

Verificar en http://localhost:8761 que los tres microservicios están registrados.

```
    # 1. Crear una mascota en ms-mascota
    POST http://localhost:8082/api/mascotas
```

```
  { "nombre":"Luna", "especie":"Gato", "raza":"Siamés",
    "edad":3, "duenoId":1 }

  # 2. Crear un veterinario en ms-veterinario
  POST http://localhost:8083/api/veterinarios
  { "nombre":"Marta", "apellido":"López",
    "especialidad":"Felinos", "email":"marta@patitas.com" }

  # 3. Crear turno → debe responder 201
  POST http://localhost:8084/api/turnos
  { "fechaHora":"2026-11-15T10:00:00",
    "motivo":"Control anual",
    "mascotaId":1, "veterinarioId":1 }
```

```
  POST http://localhost:8084/api/turnos
  { "fechaHora":"2026-11-15T10:00:00",
    "motivo":"Control", "mascotaId":999, "veterinarioId":1 }
  → 404 Not Found: { "message": "Mascota no encontrada: 999" }
```

```
  # Detener ms-mascota y enviar el mismo POST
  → 503 Service Unavailable: "Servicio remoto no disponible: ..."
```

PASO 11 — (Opcional) Circuit Breaker con Resilience4j — concepto

El manejo actual con try/catch no tiene **circuit breaker**: si ms-mascota está caído, cada llamada espera el timeout completo (5 s). Con Resilience4j se "abre" el circuito después de N fallos consecutivos y se retorna un *fallback* inmediato.

Estado	Comportamiento
CLOSED (normal)	Deja pasar las llamadas. Contabiliza fallos.
OPEN (circuito abierto)	Rechaza llamadas inmediatamente, devuelve fallback. Espera X segundos.
HALF-OPEN (prueba)	Deja pasar N llamadas de prueba. Si pasan → CLOSED; si fallan → OPEN.

Se implementará en el **Sprint 13** junto con patrones de resiliencia. Por ahora alcanza con el try/catch + timeout.

Resumen del Sprint 11

Componente	Ubicación	Responsabilidad
MascotaFeignClient	ms-turno / adapters.out.feign	Declaración HTTP hacia ms-mascota
VeterinarioFeignClient	ms-turno / adapters.out.feign	Declaración HTTP hacia ms-veterinario
MascotaFeignAdapter	ms-turno / adapters.out.feign	Implementa MascotaServicePort usando el client
VeterinarioFeignAdapter	ms-turno / adapters.out.feign	Implementa VeterinarioServicePort usando

Componente	Ubicación	Responsabilidad
		el client
MascotaServicePort	ms-turno / domain.ports.out	Puerto saliente — abstrae la comunicación
VeterinarioServicePort	ms-turno / domain.ports.out	Puerto saliente — abstrae la comunicación
TurnoServiceImpl	ms-turno / application	Orquesta validación + persistencia

Reglas importantes de este sprint

Regla 1 — Nombres Eureka exactos: El valor de `name` en `@FeignClient` debe ser igual a `spring.application.name` del servicio destino, tal como aparece en el dashboard de Eureka.

Regla 2 — No compartir clases de dominio: cada microservicio define su propia clase de respuesta reducida (DTO). Compartir un módulo common con las entidades acopla los servicios.

Regla 3 — Siempre timeout: toda llamada Feign a un servicio externo debe tener `connectTimeout` y `readTimeout` configurados.

⚠️ En producción nunca exponer /actuator sin seguridad. Se protegerá con Spring Security en el Sprint 14.

Vista previa — Sprint 12: API Gateway con Spring Cloud Gateway

Actualmente los clientes llaman a cada microservicio por su puerto (`:8081`, `:8082`, ...). Un **API Gateway** es el único punto de entrada: recibe todas las peticiones en `:8080` y las enruta dinámicamente al microservicio correcto vía Eureka. También es el lugar ideal para autenticación centralizada (JWT) y rate limiting.